

Tintero Social

Plataforma Social de Lectura

Proyecto Final DSY1103 · Arquitectura de Microservicios

Javier Escalona Padilla · Eloy Contreras
DuocUC — Ing. en Informática

Problema Abordado

- Los lectores frecuentes carecen de una plataforma integrada para organizar sus lecturas, descubrir libros recomendados por la comunidad, escribir reseñas y conectar con otros lectores.
- Las soluciones existentes son dispersas: aplicaciones para catalogar libros, redes sociales genéricas o foros sin estructura.
- No existe un espacio centralizado que combine catálogo, resenas, listas de lectura, seguimiento social y recomendaciones personalizadas.

Solución Propuesta

- Backend basado en microservicios con Spring Boot 3.4 y Spring Cloud 2024.
- 11 modulos Maven: 10 servicios de negocio + 1 API Gateway + 1 Eureka Server.
- Cada servicio con su propia base de datos aislada (Database per Service).
- Autenticacion JWT con BCrypt en user-service.
- Despliegue local con Docker Compose (22 contenedores) y remoto en Render.

Alcance Final del Sistema

Componente	Cantidad
Microservicios de negocio	10
API Gateway	1
Eureka Server	1
Endpoints REST	57
Pruebas unitarias	132 metodos @Test
Base de datos aisladas	10
Perfiles de ejecucion	3 (dev, prod, neon)
Scripts Flyway	10

Requerimientos Funcionales (54 total)

Servicio	RFs	Ejemplo
User Service	7 (RF-01 a RF-07)	Registro, login JWT, CRUD
Book Service	5 (RF-08 a RF-12)	Catalogo con filtro por titulo
Review Service	6 (RF-13 a RF-18)	Resenas, votos, edicion por autor
Reading List Service	7 (RF-19 a RF-25)	Listas personalizadas, progreso
Social Service	6 (RF-26 a RF-31)	Seguimiento, feed, estadisticas
Recommendation Service	5 (RF-32 a RF-36)	Recomendaciones, trending
Notification Service	3 (RF-37 a RF-39)	Notificaciones internas
Search Service	5 (RF-40 a RF-44)	Busqueda global con filtros
Stats Service	4 (RF-45 a RF-48)	Estadisticas y ranking
Audit Service	6 (RF-49 a RF-54)	Auditoria de acciones

Requerimientos No Funcionales (10)

RNF	Descripcion	Evidencia
RNF-01	Credenciales no expuestas en GitHub	.env.example + .gitignore
RNF-02	API Gateway centraliza peticiones	Gateway con 10 rutas lb://
RNF-03	Descubrimiento automatico	Eureka Server + 11 clientes
RNF-04	Documentacion Swagger	springdoc en 10 servicios
RNF-05	Migraciones SQL versionadas	Flyway en perfil prod
RNF-06	Perfiles por entorno	dev / prod / neon
RNF-07	Despliegue remoto	Render + Neon
RNF-08	Pruebas unitarias	132 tests en 26 archivos
RNF-09	Autenticacion JWT	JwtUtil + JwtAuthFilter
RNF-10	Contraseñas hasheadas	BCryptPasswordEncoder

Feedback y Correcciones Aplicadas

Feedback / Tarea pendiente	Correccion aplicada	Estado
Gateway sin filtros (solo predicado)	StripPrefix=1 + AddRequestHeader en 30 rutas	Corregido
Sin servidor de descubrimiento	Eureka Server + cliente en 11 servicios	Corregido
Sin migraciones SQL	Flyway con 10 scripts V1__create_*.sql	Corregido
Sin .env.example	Archivo .env.example + .gitignore	Corregido
Documentacion incompleta	README completo con 20 secciones	Corregido
Sin matriz de requerimientos	docs/matriz-requerimientos.md con 64 items	Corregido
Despliegue Render sin memoria	Profile neon configurado, requiere plan de pago	Pendiente

Arquitectura General

[Cliente HTTP]

[API Gateway :8080] ← StripPrefix + AddRequestHeader

[Eureka Server :8761] ← Descubrimiento automatico

User	Book	Revi	RL	Soc1	Reco	Noti	Sear	Stat	Audi		
:8081	:8082	:8083	:8084	:8085	:8086	:8087	:8088	:8089	:8090		

[BDs aisladas: MySQL / PostgreSQL / H2]

- Patron Database por Service: cada servicio tiene su propia BD.
- 3 perfiles: dev (H2), prod (MySQL + Flyway), neon (PostgreSQL + Render).
- Gateway resuelve servicios via lb:// en perfiles dev/prod.
- En neon, Gateway usa URLs directas (single container en Render).

Microservicios Implementados

Servicio	Puerto	Responsabilidad
eureka-server	8761	Descubrimiento automatico (Netflix Eureka)
gateway-service	8080	API Gateway, enrutamiento, StripPrefix, headers
user-service	8081	Registro, login, JWT, perfiles de usuario
book-service	8082	Catalogo de libros, ISBN unico
review-service	8083	Resenas, votos, edicion solo por autor
reading-list-service	8084	Listas personales, progreso de lectura
social-service	8085	Seguimiento, feed de actividad
recommendation-service	8086	Recomendaciones personalizadas, trending
notification-service	8087	Notificaciones internas a usuarios
search-service	8088	Busqueda global de libros y usuarios
stats-service	8089	Estadisticas de lectura y ranking
audit-service	8090	Registro de auditoria de acciones

Modelo de Datos (14 tablas en 10 BDs)

BD del Servicio	Tabla(s)	Entidad principal
db_users	users	User (id, name, email, passwordHash, bio, createdAt)
db_books	books	Book (id, title, author, isbn, genre, synopsis, pubYear)
db_reviews	reviews, votes	Review + Vote (rating, comment)
db_reading_lists	reading_lists, reading_list_items	ReadingList + Item (progress 0-100%)
db_social	follows	Follow (followerId, followedId)
db_recommendations	recommendations, genre_preferences	Recommendation + GenrePreference
db_notifications	notifications	Notification (userId, message, read, senderId)
db_searchs	book_index, user_index	Indices de busqueda
db_stats	stat_snapshots	StatSnapshot (totalBooksRead, avgRating)
db_audit	audit_event	AuditEvent (eventType, resourceType, resourceId)

Flujo Funcional Principal

- 1. Usuario se registra: POST /api/users/register → user-service (BCrypt hash, JWT)
- 2. Usuario inicia sesion: POST /api/users/login → user-service → retorna token JWT
- 3. Busca libros: GET /api/search?q=titulo → search-service → retorna coincidencias
- 4. Agrega libro a lista: POST /api/lists/{id}/books → reading-list-service
- 5. Escribe resena: POST /api/reviews → review-service (valida autor)
- 6. Sigue a otro usuario: POST /api/social/follow → social-service
- 7. Ve feed de actividad: GET /api/social/feed/{userId} → social-service (agrega reviews + listas de seguidos)
- 8. Recibe recomendaciones: GET /api/recommendations/{userId} → recommendation-service
- 9. Consulta estadísticas: GET /api/stats/user/{userId} → stats-service
- 10. Evento registrado: POST /api/audit/event → audit-service

Flujo Tecnico entre Capas y Servicios

[Cliente] → HTTP Request → [API Gateway :8080]

- └ StripPrefix=1 (elimina /api)
- └ AddRequestHeader=X-Forwarded-Prefix
- └ Resolve lb://service-name via Eureka

→ [Microservicio destino :808x]

- └ Controller → Service → Repository → JPA → BD aislada
- └ GlobalExceptionHandler (@ControllerAdvice)
- └ @Slf4j (logger en services y clients)

→ Response JSON → [Cliente]

[Eureka Server :8761] ← registro automatico de cada servicio al arrancar

Seguridad y Roles

- Autenticación basada en JWT (Json Web Token).
- BCryptPasswordEncoder para hash de contraseñas (nunca texto plano).
- JwtAuthFilter registrado antes de UsernamePasswordAuthenticationFilter.
- Endpoints públicos: register, login, validate, GET users, Swagger UI.
- Endpoints protegidos: todos los demás requieren token válido.
- Roles: el sistema actualmente no distingue roles (todos son usuarios regulares). No hay endpoints de admin.
- CSRF deshabilitado para APIs REST (estado sin sesión).
- H2 Console accesible solo en perfil dev.

Pruebas Realizadas

Metrica	Valor
Archivos de test	26
Metodos @Test	132
Servicios con tests	10 (todos excepto gateway y eureka)
Frameworks	JUnit 5 + Mockito
Ejecucion	./mvnw test

Cobertura por servicio:

Servicio	Tests
user-service	UserServiceTest, JwtUtilTest
book-service	BookServiceTest
review-service	ReviewServiceTest
reading-list-service	ReadingListServiceTest
social-service	SocialServiceTest
recommendation-service	RecommendationServiceTest
notification-service	NotificationServiceTest
search-service	SearchServiceTest
stats-service	StatsServiceTest

Documentacion Swagger / OpenAPI

- Cada microservicio (excepto gateway y eureka) expone documentacion interactiva via springdoc-openapi.
- URL local: `http://localhost:{puerto}/swagger-ui/index.html`
- Configuracion centralizada en `OpenApiConfig.java` por servicio.
- Permite probar endpoints directamente desde el navegador.
- 10 servicios documentados con OpenAPI 3.0.

Servicio	Swagger URL
user-service	<code>http://localhost:8081/swagger-ui/index.html</code>
book-service	<code>http://localhost:8082/swagger-ui/index.html</code>
review-service	<code>http://localhost:8083/swagger-ui/index.html</code>
reading-list-service	<code>http://localhost:8084/swagger-ui/index.html</code>
social-service	<code>http://localhost:8085/swagger-ui/index.html</code>
recommendation-service	<code>http://localhost:8086/swagger-ui/index.html</code>
notification-service	<code>http://localhost:8087/swagger-ui/index.html</code>
search-service	<code>http://localhost:8088/swagger-ui/index.html</code>
stats-service	<code>http://localhost:8089/swagger-ui/index.html</code>
audit-service	<code>http://localhost:8090/swagger-ui/index.html</code>

Despliegue Local

Opcion 1 — Ejecucion directa:

- Requisitos: Java 21+, Maven 3.9+ (o mvnw incluido).
- Orden de arranque: Eureka Server primero → Microservicios → Gateway al final.
- Perfil dev: H2 en memoria, sin Flyway.
- Comando: `./mvnw spring-boot:run -pl eureka-server`

Opcion 2 — Docker Compose:

- 22 contenedores: 10 MySQL + 1 Eureka + 10 microservicios + 1 Gateway.
- Comando: `docker compose up --build`
- Perfil prod: MySQL + Flyway activado.

Despliegue Remoto — Render + Neon

URL publica: <https://tintero-social.onrender.com>

Servicio	Puerto	Estado en Render
gateway-service	8080	Desplegado (unico puerto expuesto)
user-service	8081	Desplegado
book-service	8082	Desplegado
review-service	8083	Desplegado
reading-list-service	8084	Desplegado
social-service	8085	Desplegado
notification-service	8087	No desplegado (memoria)
recommendation-service	8086	No desplegado (memoria)
search-service	8088	No desplegado (memoria)
stats-service	8089	No desplegado (memoria)
audit-service	8090	No desplegado (memoria)

- Render plan gratuito: 512 MB de memoria. Hicimos la prueba con todos los servicios activos pero no tuvimos buen resultado, luego con 6 servicios y no hubo caso, llegábamos al límite de memoria y el deploy arroja error.
- Base de datos utilizada: Neon (PostgreSQL serverless). Profile neon: Eureka desactivado, URLs directas.

Principales Dificultades Tecnicas

Dificultad	Solucion
Gateway sin filtros configurados	StripPrefix=1 y AddRequestHeader en las 30 rutas
Sin mecanismo de descubrimiento	Eureka Server + cliente en 11 servicios
Sin migraciones SQL	Flyway con 10 scripts V1__create_*.sql
Credenciales expuestas en repositorio	.env.example + .gitignore excluye .env
README no cumplia pauta completa	Reescrito con 20 secciones obligatorias
Render con memoria insuficiente	Profile neon optimizado (Eureka off, 6 servicios)
Servicios excluidos de Render	Funcionan localmente con docker compose

Distribucion de Trabajo del Equipo

Miembro	Responsabilidades principales
Javier Escalona Padilla	Arquitectura del proyecto, Eureka Server, Gateway, Docker Compose, despliegue Render, pruebas unitarias, documentacion tecnica
Eloy Contreras	Desarrollo de microservicios (user, book, review, reading-list, social, notification, search, stats, audit), configuracion YAML, documentacion Swagger

- Ambos colaboramos en la revisión y en la correccion de bugs.
- Seguimiento de tareas via GitHub, en respectivos commits.

Tintero Social

54 requerimientos funcionales implementados

10 requerimientos no funcionales cumplidos

132 pruebas unitarias verificadas

Arquitectura de microservicios con Spring Boot 3.4 + Spring Cloud 2024

Despliegue local (Docker) e intento remoto (Render + Neon)

URL: <https://tintero-social.onrender.com>

Gracias — Tintero Social · DSY1103 · DuocUC